

第八届中国研究生人工智能创新大赛

Sentinel-Guardian

企业 AI Agent 上线前自主安全审计与自动加固智能体

项目文档

版本号码: v1.0

日 期: 2026.09.01

团队名称: TITAN

参赛组别: 研究生组

目录

记录更改历史	III
1 项目概况	1
1.1 背景和基础	1
1.2 场景和价值	1
1.3 所需支持	2
2 项目规划	2
2.1 整体目标	2
2.2 技术创新点	2
3 实施方案	3
3.1 技术可行性分析	3
3.2 技术细节	4
3.3 计划和分工	5
4 参考资料	5

记录更改历史

序号	更改原因	版本	作者	更改日期	备注
1	项目立项、需求分析与技术调研初稿	v0.1	陈中泽、曹天硕	2026.07.05	项目启动
2	完成整体架构设计与技术选型	v0.2	曹天硕、陈中泽	2026.07.18	C0 阶段
3	核心画像模块原型：Docker 镜像离线解析	v0.3	陈中泽	2026.07.30	C1 阶段
4	攻击引擎第一版：7 类威胁与升级阶梯	v0.4	曹天硕	2026.08.10	C2 阶段
5	防御模块与 baseline/guarded 闭环编排	v0.5	陈中泽、曹天硕	2026.08.18	C3 阶段
6	企业知识助手案例与真实容器审计验证	v0.6	曹天硕	2026.08.24	C4 阶段
7	Windows 平台适配与 Benchmark 内化	v0.7	陈中泽	2026.08.28	C5 阶段
8	全量测试通过与工程质量收尾	v0.9	陈中泽、曹天硕	2026.08.31	冻结代码
9	初赛正式提交版本	v1.0	陈中泽、曹天硕	2026.09.01	C6 提交

1 项目概况

1.1 背景和基础

随着以大语言模型（LLM）为核心的 AI Agent 在企业中大规模落地，上线前的安全验收成为亟待解决的工程问题。现有安全实践存在三大痛点：一是缺乏自动化安全验收机制，企业依赖人工代码审查，无法系统覆盖 Prompt Injection、工具滥用、记忆污染等 LLM Agent 特有攻击向量；二是攻击与防御评估割裂，现有工具（PyRIT、Garak 等）仅做红队攻击或防御检测之一，缺少完整闭环；三是缺乏可审计的上线决策依据，无法自动输出带证据绑定的上线结论。

项目灵感来源于：如何让一个智能体自主完成另一个智能体的安全验收？基于这一核心命题，团队系统调研了 AgentDojo（NeurIPS 2024）、InjecAgent、AgentHarm 等主流 Agent 安全 Benchmark，梳理了 N1-N8 共 8 个 Agent 流水线攻击节点，完成了从镜像解析、自动攻击、防御生成到上线决策的完整技术路径验证。

已有工程基础:1,068 个 Python 自动化测试全部通过;在企业知识助手（RAG+Tool Use）和 OpenManus（MCP 多 Agent）两类真实 Agent 上完成完整闭环审计，无防御 ASR 达 98.1%，加固后降至 12.4%，误报率（FPR）保持 0%。

团队构成：TITAN 团队成员专注于 AI Agent 安全、大语言模型应用与系统工程方向，具备从安全研究到全栈工程的完整能力覆盖，参赛组别为研究生组。

1.2 场景和价值

应用场景：Sentinel-Guardian 主要面向三类场景：

- **企业 Agent 上线安全验收**：企业将 AI Agent（智能客服、知识助手、自动化运维等）推向生产前，提交 Docker 镜像，系统自动执行安全审计并输出可审计的四态上线决策，满足 AI 安全合规要求；
- **AI 红队与漏洞发现**：安全研究人员对目标 Agent 实施自动化渗透测试，覆盖 7 类威胁，攻击策略自动升级直至突破或确认防御有效；
- **防御效果持续验证**：对已部署 Guard 的 Agent 持续执行 baseline/guarded 配对测试，量化防御收益。

市场调研与对比分析：据 Gartner 预测，至 2026 年企业级 AI Agent 部署将增长 300%。欧盟 AI 法案及国内 AI 安全治理规范对 AI 系统安全合规要求趋严，AI Agent 安全审计将成为刚性需求。如表 1 所示，现有工具均为单点工具，Sentinel-Guardian 是首个提供”画像 → 攻击 → 防御 → 决策”完整闭环的自主审计系统。

Table 1: 与现有工具对比分析

工具	画像提取	自动攻击	防御生成	上线决策	闭环复测	证据绑定
PyRIT（微软）	否	是	否	否	否	否
Garak	否	是	否	否	否	否
AgentDojo	否	是	否	否	否	否
LLM Guard	否	否	是	否	否	否
Sentinel-Guardian	是	是	是	是	是	是

潜在社会价值：为企业 AI 治理提供自动化、可溯源的安全验收基础设施，降低 AI Agent 上线风险，助力 AI 安全合规落地。

1.3 所需支持

算力支持：

- 离线画像与测试（无需 GPU）：普通 CPU 机器（Python 3.10+，8GB RAM）即可完成；
- 真实审计执行：需 Docker Desktop（本地沙箱）运行被审计 Agent 容器；
- LLM 调用：攻击规划、防御生成和语义补全需模型 API（OpenAI / Anthropic / 国产大模型），支持三组独立凭据配置（target/attack/defense 模型解耦），API Key 仅存进程内存，不持久化。

硬件支持：桌面应用支持 macOS arm64 和 Windows x86_64 双平台；真实容器运行需本地安装 Docker Desktop。

相关培训：需了解 AI Agent 安全基础概念（Prompt Injection、工具调用安全）、Docker 基本操作及 LangChain/LangGraph 框架。

2 项目规划

2.1 整体目标

参赛期间本项目的整体目标是：**构建并验证一个完整可用的企业 AI Agent 上线前自主安全审计与自动加固原型系统，并在真实 Agent 场景中完成端到端验证。**

可展示原型系统（已实现）：桌面应用（macOS arm64 + Windows x86_64）具备完整 UI，支持 Agent 接入、审计触发、实时进度、结果可视化和报告导出；系统设计为通用”任意 Agent 检测平台”，通过标准 agent.json + image.tar 接入任意企业 AI Agent。

行业场景初步验证（已实现）：针对两类真实企业 Agent——**电商客服 Agent**（RAG + Tool Use + 记忆持久化）和 **OpenManus Agent**（MCP 协议 + 多 Agent 协作）——完成完整审计闭环。核心验证指标如表 2 所示。

Table 2: 核心验证指标		
指标	目标值	实测值
无防御攻击成功率（ASR）	≥ 80%	98.1% （103/105）
加固后 ASR	≤ 20%	12.4%
ASR 下降幅度	≥ 60pp	85.7pp
误报率（FPR）	= 0%	0%
正常业务效用（Clean Utility）	= 100%	100%
OpenManus 真实容器成功率	15/15	15/15
baseline/guarded 配对	5/5	5/5

2.2 技术创新点

通过对 PyRIT、Garak、AgentDojo、LLM Guard 等相关工作的对比调研，本项目主要技术创新点如下：

创新点 1：离线镜像解析生成证据闭合 Agent 画像

现有工具以”黑盒”方式访问目标 Agent，无法了解内部结构，导致攻击盲目、防御无据。本项目无需运行 Docker 容器，仅通过静态解析镜像即可提取完整安全画像，采用五阶段流水线：static_facts（入口/依赖/权限）→ framework_detect

(框架识别) → dataflow (数据流图重建) → semantic_enrich (LLM 语义标注) → dynamic_verify (动态探针验证)。画像内容包含 Agent 节点图 (N1-N8 流水线)、工具列表与风险等级、权限范围、RAG 检索链、MCP 服务器注册等, 所有证据绑定 image_digest + profile_sha256, 采用 STRIDE + MITRE ATLAS 双框架标注。

创新点 2: 画像驱动的自适应攻击战役

AgentDojo 等 Benchmark 使用固定测试集, 无法针对特定 Agent 拓扑结构定制攻击, 也不具备反思-升级能力。本项目实现三项突破: (1) **N1-N8 全节点覆盖**: 将 Agent 流水线分为 8 个攻击节点, 每个攻击精确标注目标节点, 相比 AgentDojo 仅覆盖 N1/N3/N4/N7, 本系统实现全节点覆盖; (2) **Benchmark 攻击手法内化**: 内化了 AgentDojo 6 种攻击手法 (NeurIPS 2024 最高 ASR 手法)、InjecAgent 30 条 tool_response 注入样本、AgentHarm 25 条有害行为 payload, 作为升级阶梯的最高强度台阶; (3) **反思-升级阶梯**: 7 类威胁各配备多级策略阶梯, 失败后 LLM 自主反思并升级, 攻击力度自动增强。

创新点 3: Schema 约束的 LLM 自主规划与确定性 Policy 校验

采用 Pydantic schema 强约束 LLM 生成的攻击计划, 配合确定性 Policy 层校验授权范围、危险动作边界和预算合法性; LLM 输出无效或越权时自动回退并披露原因; 每轮新攻击集在执行前需用户明确批准 (HITL 授权门禁)。

创新点 4: 局部 Guard 自动生成与同条件配对复测

Defense Agent 基于失效节点生成局部 Guard, 精确挂载到攻击成功的具体节点; 采用三层防御体系 (IO 过滤、上下文隔离、行为监测); baseline/guarded 配对绑定同一 source_snapshot_sha256、同一模型/case/seed/预算, 确保对比有效性。goal_drift 场景引入 HITL 兜底 (asked=9, benign ask=0, 无正常业务误伤)。

创新点 5: 完整可溯源证据闭环与四态上线决策

每次审计绑定 image_digest + profile_sha256, 输出完整证据包 (agent-profile-v0.2、baseline/guarded 对比结果、失效节点归因、Guard 挂载记录、bundle-manifest、evidence-index、原始轨迹); 四态决策 (允许上线 / 修复后复测 / 高风险禁止上线 / 需人工审批) 可追溯到具体证据文件, 支持离线复核。

3 实施方案

3.1 技术可行性分析

数据采集:

- **Agent 画像数据 (静态解析)**: 无需外部数据集, 通过解析 Docker 镜像文件系统 (manifest、config、layer tar) 自动提取, 已在两类真实 Agent 上验证完整性;
- **攻击 Payload 数据 (Benchmark 内化)**: 内化三个公开 Benchmark 的攻击手法 (均为 MIT License, 已严格标注来源): AgentDojo (NeurIPS 2024, 7 条代表性 payload)、InjecAgent (精选 30 条 tool_response 注入样本, 覆盖 N3/N4/N5/N7/N8)、AgentHarm (精选 25 条, 覆盖 7 类有害类别);
- **评测测试集 (合成)**: 攻击场景 7 类 × 15 条 = 105 条, 良性基准 42 条, 全部使用合成虚构数据, 不含真实 PII。

行业知识获取: 采用 STRIDE 威胁建模框架 (Microsoft) 和 MITRE ATLAS 技术编号, 均为公开标准, 对每类威胁给出 ATLAS 技术编号和 STRIDE 分类标注。

算力与硬件可行性: 离线全量测试 (1,068 tests) 在普通笔记本 5 分钟内完成, 无 GPU 需求; 离线演示审计无需 Docker 和 API Key; 真实容器审计 (OpenManus

W2 rerun10) 共 15 次均在合理时间内完成；桌面应用打包已在 macOS arm64 和 Windows x86_64 两个平台验证通过。

3.2 技术细节

系统整体架构（三层）：

控制面（Control Layer）
JWT 认证与租户隔离
Agent 资产索引（增量扫描 agents/ 目录）
静态画像工作流（5 阶段状态机）
多轮审计状态机：
 created -> profiling -> planning -> baseline_execution
 -> evaluation -> defense_generation -> guarded_execution
 -> decision -> completed / needs_approval / failed

决策层（Intelligence Layer）
Attack Agent: 读取画像 -> LLM 生成受 schema 约束的攻击计划 -> 反思-升级
Evaluation Agent: 确定性 Oracle + 轨迹信号 + 节点归因 -> ASR/FPR/
 Utility
Defense Agent: 基于失效节点生成局部 Guard -> 三层防御配置

隔离执行面（Runtime Layer）
Docker Desktop 沙箱（不使用客户生产环境）
OpenManus Runtime（MCP 协议执行器）
被测 Agent 镜像副本（完全隔离）+ 局部 Guard 挂载点

Agent 画像提取阶段：

Table 3: 画像提取五阶段流水线		
阶段	核心操作	输出
static_facts	AST 解析入口/依赖/权限/工具签名	节点列表、工具表、权限声明
framework_detect	框架识别（LangChain / OpenManus / MCP）	框架标签、Agent 拓扑
dataflow	重建用户输入到高风险 Sink 数据流	风险路径、risk_level
semantic_enrich	LLM 语义补全 STRIDE 分类（可选）	自然语言风险描述
dynamic_verify	运行 N1-N8 探针场景（可选）	事件流（tool_called / guard_decision）

安全评估指标（7 类威胁，105 cases + 42 benign）：

工程质量指标：

技术栈：后端 FastAPI + Uvicorn（Python 3.10+）；前端 React 18 + TypeScript + Vite；桌面壳 pywebview（macOS cocoa / Windows edgechromium）；LLM 框架 LangChain + LangGraph；向量检索 ChromaDB + sentence-transformers；容器运行时 Docker SDK；数据验证 Pydantic v2；测试 pytest + vitest + Playwright。

Table 4: 分威胁类别安全评估结果

威胁类别	Cases	无防御 ASR	加固后 ASR	FPR
jailbreak	15	100%	53.3%	0%
training_data_leakage	15	86.7%	0%	0%
environment_awareness_poll.	15	100%	0%	0%
prompt_injection	15	100%	33.3%	0%
tool_tampering	15	100%	0%	0%
memory_poisoning	15	100%	0%	0%
goal_drift	15	100%	0%	0%
综合	105	98.1%	12.4%	0%

Table 5: 工程质量指标

指标	数值
Python 自动化测试	1,068 passed, 2 deselected
前端自动化测试	50 passed
Playwright e2e 视口测试	2 passed (1440px 桌面 + 390px 移动)
WCAG 可访问性	0 violations
npm 安全审计	0 vulnerabilities

3.3 计划和分工

整体计划（结合参赛时间点）：

Table 6: 项目整体计划

阶段	时间	主要工作	状态
C0	前期	口径统一、架构设计、核心协议定义	完成
C1	前期	AuditTask / AuditPlan / ReleaseDecision schema 定义	完成
C2	前期	LLM 受约束规划、可视化计划页面	完成
C3	前期	多轮状态机、baseline/guarded 配对测试	完成
C4	前期	企业知识助手案例、四类攻击、局部 Guard	完成
C5	前期	完整 UI、桌面打包（macOS + Windows）	完成
C6	2026.09.01	初赛材料提交	进行中

团队分工：

4 参考资料

[1] Debenedetti E. et al., “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” NeurIPS 2024 D&B Track, arXiv:2406.13352. <https://github.com/ethz-spylab/agentdojo>

[2] Zhan Q. et al., “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents,” arXiv:2403.02691. <https://github.com/uiuc-kang-lab/InjecAgent>

Table 7: 团队分工

分工方向	主要职责
系统架构 & 后端	FastAPI 服务、审计状态机、证据存储、画像 workflow 设计
攻击引擎	Attack Agent、反思-升级阶梯、Benchmark 攻击手法内化
防御引擎	Defense Agent、三层 Guard 体系、baseline/guarded 复测框架
运行时 & 容器	Docker 隔离执行器、OpenManus Runtime 集成、动态探针
前端 & 产品	React UI、轨迹可视化、审计工作区、桌面应用双平台打包
安全研究	威胁建模 (STRIDE/ATLAS)、Benchmark 调研、测试用例设计

[3] Andriushchenko M. et al., “AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents,” arXiv:2410.09024. <https://huggingface.co/datasets/ai-safety-institute/AgentHarm>

[4] Zhang J. et al., “Agent-SafetyBench: Evaluating the Safety of LLM Agents,” 2024. <https://github.com/thu-coai/Agent-SafetyBench>

[5] Yi J. et al., “Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models,” arXiv:2312.14197. <https://github.com/microsoft/BIPIA>

[6] Microsoft AI Red Team, “PyRIT: Python Risk Identification Tool for Generative AI.” <https://github.com/Azure/PyRIT>

[7] MITRE Corporation, “ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems.” <https://atlas.mitre.org/>

[8] LangChain AI, “LangGraph: Building stateful, multi-actor applications with LLMs.” <https://github.com/langchain-ai/langgraph>

[9] Microsoft Security Development Lifecycle, “Threat Modeling with STRIDE.” <https://learn.microsoft.com/azure/security/develop/threat-modeling-tool-threats>

[10] Gartner, “Predicts 2025: Agentic AI Will Transform Customer Service Delivery,” 2024.

[11] European Parliament, “Regulation on Artificial Intelligence (AI Act),” 2024.